

Adaptive Auto-Harness: Sustained Self-Improvement for Agentic System Deployment on Open-Ended Task Streams

Anonymous ACL submission

Abstract

Auto-harness systems such as A-Evolve, GEPA, and Meta-Harness automatically construct an LLM agent’s harness (prompts, skills, [Bing: memories], tools, infrastructure) and report strong gains on static benchmarks. Real-world deployment, however, demands *sustained agentic system optimization*: continual improvement of the harness as the agent operates on an unbounded stream of tasks. We identify three deployment dimensions that prior systems overlook and propose an analytical framework, a regret decomposition $L_{\text{evo}} + L_{\text{adapt}}$, that derives two complementary operations: *continual auto-harness* (a stateful multi-agent evolver with cross-cycle knowledge) and *runtime adaptation* (a strategy-tree navigator that selects a task-relevant harness prior to solving), plus a human-in-the-loop channel for regimes outside prior experience. Our system outperforms A-Evolve, GEPA, and Meta-Harness on three real task streams. #Zewen: will revise in the end#

1 Introduction

An agent’s harness, comprising the prompts, skills, tools, and supporting infrastructure that surround a fixed LLM, is a primary determinant of task-solving performance. *Auto-harness systems* such as A-Evolve (Lin et al., 2026), GEPA (Agrawal et al., 2025), and Meta-Harness (Lee et al., 2026) performs evolution to construct harness automatically from execution feedback, and report substantial gains on static offline benchmarks such as SWE-bench (Jimenez et al., 2024). However, online real-world deployment, whose tasks and evaluations follow a strict chronologic order, demands *sustained self-improvement*: a sustained improvement of the harness as the agent operates on an open-ended task stream rather than on a fixed evaluation set. Representative settings include prediction-market platforms with thousands of questions across diverse topics over weeks (Cheng et al., 2026), secu-

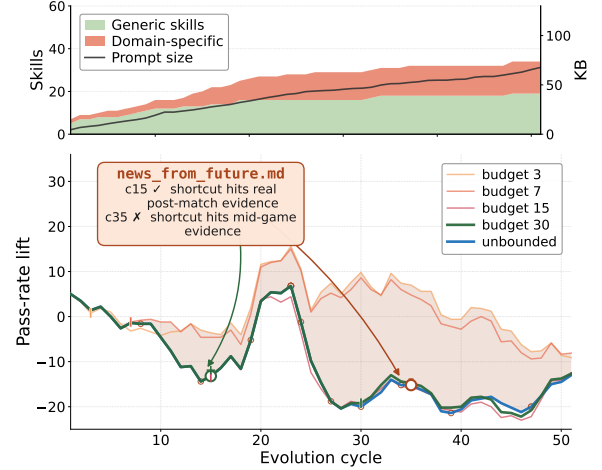


Figure 1: Harness overfits past-stream patterns. **Top:** unbounded-run skills grow from 12 generic to 34, prompt size grows from 2KB to 68KB. **Bottom:** pass-rate lift for evolution with fixed cycles on Polymarket trading tasks overtime.

rity competitions spanning over a decade of evolving challenges (Zhuo et al., 2025), and forecasting services handling cross-lingual queries from heterogeneous sources (Zeng et al., 2025).

A concrete deployment failure illustrates the gap. As shown in Figure 1, we perform A-evolve on polymarket trading tasks across weeks with different evolution budgets (evolution stops after N cycles). While evolution initially improves the harness (lift compared to no evolution), the runs with more budget accumulates source preferences and tools growth that consume context and bias later tasks. Across all five evolution-budget settings, every curve peaks early and declines, with fewer budget runs beating those with more evolutions, which indicates achieving sustained improvement on open-ended task stream requires more than continual update with more cycles.

Therefore, as illustrated in Figure 2, we point out three deployment dimensions that distinguish the two settings. **(D1) Unbounded Streams.** The

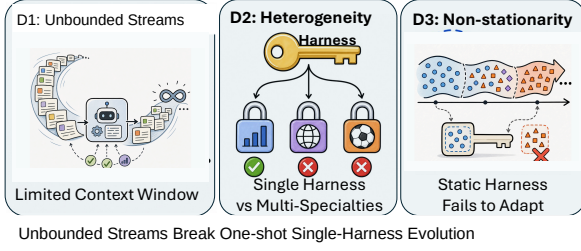


Figure 2: Three deployment dimensions in open-ended task streams.

task stream has no fixed train/test cutoff or designated endpoint; feedback, trajectories, and harness state keep accumulating during deployment, creating heavy burden for the evolution. Existing auto-harness systems built around a single-agent evolver are bounded by a finite context window, leaving little room to approach the large and growing volume of trajectories that effective and generalizable harness would require to be built. **(D2) Task heterogeneity.** The stream contains tasks of varied types that demand different harness components. A prediction-market platform, for example, mixes politics, sports, and finance questions in the same hour, each calling for distinct sources, tools, and prompting patterns. This rationale parallels a more general phenomenon: a single fixed policy is rarely optimal across heterogeneous problems (Miao et al., 2025). Existing auto-harness systems analogously deploy a static dense harness across the stream, with no solve-time adaptation to the task at hand. **(D3) Distributional non-stationarity.** As the stream progresses, incoming tasks shift away from the experience the harness was last fitted on. A harness optimised for recent cycles therefore drifts out of fit for new tasks, even with rich historical experience. Closing this gap requires per-task contextual adaptation of the harness, not only continued historical fitting. Additional diagnostics are shown in Appendix A.

We address these three dimensions under a unified analytical framework and identify two root gaps that prior auto-harness systems overlook. This analysis motivates: (1) *Sustained auto-harness* (§3.3), which replaces stateless one-shot evolution with a stateful multi-agent system with cross-cycle knowledge for better harness construction; and (2) *Solve-time adaptation* (§3.4), which adapts a task-relevant harness for each problem prior to solving, restoring per-task fit on a heterogeneous, drifting stream. Beyond the two axes, we further introduce a third axis: human-in-the-loop (HITL) for auxil-

iary steering of the harness to incorporate human insights and provisions that are absent from historical experience. Our contributions are:

- (1) **Deployment-regime analysis of open-ended agentic streams.** We formalise why static auto-harness evaluation is insufficient once tasks arrive as unbounded, heterogeneous, and non-stationary streams. The framework decomposes the gap towards the optimal harness into evolution loss and adaptation loss, providing guidance for auto-harness system designs.
- (2) **Adaptive Auto-harness system.** We introduce a stateful multi-agent evolver for sustained harness construction, a harness-tree router for solve-time adaptation, and structurally triggered human-in-the-loop hooks for regimes where historical experience is insufficient.
- (3) **Comprehensive empirical validation and diagnosis.** We evaluate on three temporal streams spanning prediction markets, security challenges, and event forecasting against other auto-harness systems. Beyond aggregated performance, we provide in-depth analysis and evidence for the proposed gaps, component ablations, and human-in-the-loop slice analyses.

2 Related Work

Continual Learning in Task Streams. Continual learning studies systems that learn from a sequence of tasks while retaining earlier capabilities (Buzzega et al., 2020; Wang et al., 2022b,a). Domain and test-time adaptation address distribution shift (Ben-David et al., 2010; Ganin et al., 2016; Wang et al., 2021; Liang et al., 2020), and mixture-of-experts methods route heterogeneous inputs to specialised components (Jacobs et al., 1991; Shazeer et al., 2017). These directions cover pieces of our D1–D3 setting, but they usually adapt model weights, classifiers, or expert modules. We study the harness-level analogue: prompts, skills, tools, memory, and infrastructure must be constructed from accumulated trajectories and selected for each incoming task.

Self-improving and self-evolving agents. The closest LLM-agent precedents to our setting are systems that update the harness directly from execution feedback. A-Evolve (Lin et al., 2026) introduces a linear-chain evolver: each cycle reads batch trajectories and mutates prompts, skills, memory, and tools. GEPA (Agrawal et al., 2025) adds reflective Pareto prompt evolution using textual feedback

rather than scalar reward. Meta-Harness (Lee et al., 2026) uses a growing filesystem archive and Claude Code as the proposer. Continual Harness (Karten et al., 2026) enables online adaptation within a single continuous deployment run through alternating action/refinement cycles. SkillOS (Ouyang et al., 2026) learns a skill curation policy via reinforcement learning, training a curator to select and refine reusable skills from repeated interactions.

3 Method

3.1 Problem Formulation

We consider tasks arriving in an open-ended stream $x_1, x_2, \dots, x_T$ with $x_t \sim P_t$, where P_t is the task distribution at time t and each x_t has a fixed ground truth $y(x_t)$. The agent observes the history of prior experience $\mathcal{H}_t = \{(x_i, r_i, \tau_i)\}_{i=1}^{t-1}$, where r_i is optionally the realized reward on task x_i and τ_i is the agent’s solving trajectory (actions, intermediate observations, tool calls). A *harness* $C = \varphi(\mathcal{H}_t)$, with $|C| \leq K$, is a bounded representation (prompts, skills, memory, and tools) with capacity budget K , where φ is the *evolver*: the agentic system that automatically transforms historic experience into the harness. A solver agent then acts via the policy $a_t \sim \pi(a \mid x_t, C_t)$, where π is the LLM-induced sampling distribution over actions conditioned on the task and harness.

3.2 Analytical Framing

The three deployment dimensions of §1 surface concrete failures, but they do not by themselves identify what an evolver should fix. To pinpoint the root causes, we frame the problem analytically: we cast harness construction as a regret-minimization problem against an oracle reference, and decompose the regret into two complementary loss terms that map directly to actionable design axes.

Utility and regret. We define the utility of a harness C on a task x_t as

$$V(C, x_t) = \mathbb{E}_{a \sim \pi(\cdot \mid x_t, C)}[r(a, y(x_t))], \quad (1)$$

which is the expected reward of the harness-conditioned solver. The full-history utility is the corresponding ceiling under the capacity budget, $V(\mathcal{H}_t, x_t) := \sup_{C: |C| \leq K} V(C, x_t)$, that is, the best utility attainable by any bounded harness on x_t . The regret of the evolver-constructed harness is then

$$\text{Regret}(\varphi, x_t) = V(\mathcal{H}_t, x_t) - V(\varphi(\mathcal{H}_t), x_t) \geq 0, \quad (2)$$

non-negative since $\varphi(\mathcal{H}_t)$ is one element of the supremum’s domain. Operationally, this ceiling matches what a solver granted $\mathcal{H}_t$ directly, under the same compute and tool budget as φ , could attain by reconstructing any candidate harness on the fly. **Solve-time optimal harness.** Fix the evolver class Φ and define the solve-time optimal harness for a particular task:

$$C_\Phi^*(x_t) = \arg \max_{\substack{C = \varphi(\mathcal{H}_t, x_t), \varphi \in \Phi \\ |C| \leq K}} V(C, x_t). \quad (3)$$

This is an oracle reference: it is the best harness the evolver class Φ could produce if it were allowed to condition on the incoming task x_t at solve time. A deployed evolver φ commits to one harness $\varphi(\mathcal{H}_t)$ before x_t is observed; using C_Φ^* as pivot between $V(\mathcal{H}_t, x_t)$ and $V(\varphi(\mathcal{H}_t), x_t)$ yields the following decomposition.

Proposition 1 (Regret Decomposition). *For any deployed evolver $\varphi \in \Phi$,*

$$\mathbb{E}_{x_t}[\text{Regret}(\varphi, x_t)] = L_{\text{evo}}(\Phi) + L_{\text{adapt}}(\varphi), \quad (4)$$

where

$$L_{\text{evo}}(\Phi) = \mathbb{E}_{x_t}[V(\mathcal{H}_t, x_t) - V(C_\Phi^*(x_t), x_t)], \quad (5)$$

$$L_{\text{adapt}}(\varphi) = \mathbb{E}_{x_t}[V(C_\Phi^*(x_t), x_t) - V(\varphi(\mathcal{H}_t), x_t)]. \quad (6)$$

L_{evo} is the *evolution loss*: it reflects the evolver class Φ ’s capability gap, regardless of how many cycles the evolver runs. A single-agent prompt editor cannot produce multi-file infrastructure; that ceiling is structural, not a matter of effort. Reducing L_{evo} therefore requires pursuing more capable evolver systems with access to broader control and feedbacks. L_{adapt} is the *adaptation loss*: the oracle builds the optimal harness per task, but a deployed φ commits to one harness before seeing x_t . Therefore, even with an optimal evolver system, L_{adapt} exists as long as task heterogeneity persists.

Human-in-the-loop as a third axis. The decomposition assumes $\mathcal{H}_t$ contains relevant signal for x_t . When it does not, the regret decomposition no longer applies; we address this case via a third axis outside $L_{\text{evo}} + L_{\text{adapt}}$, the human-in-the-loop channel (§3.5).

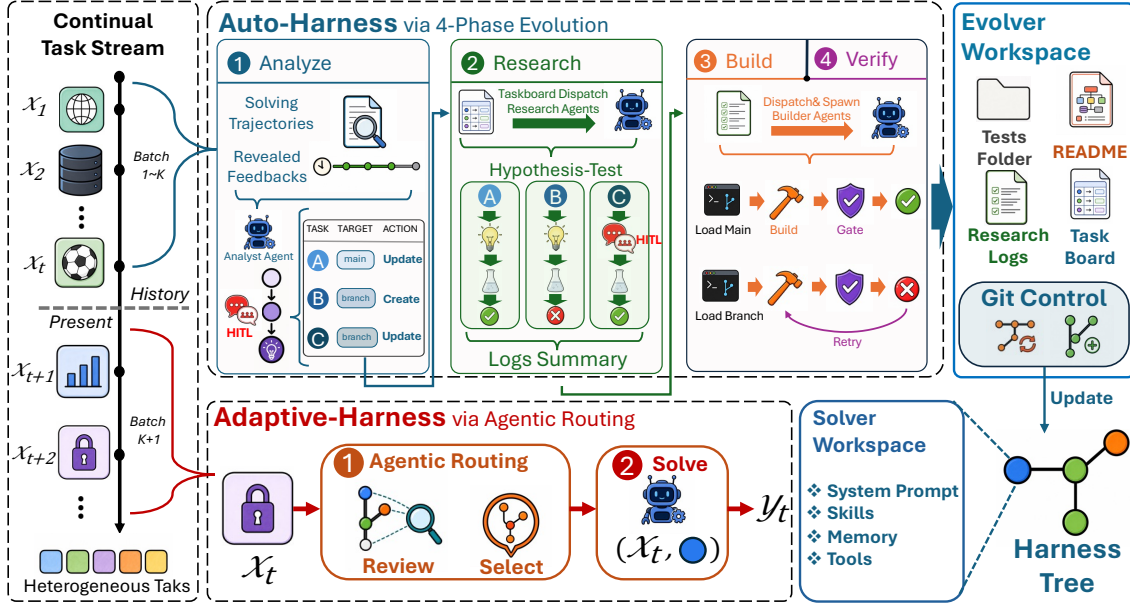


Figure 3: Overview of the Adaptive Auto-Harness system. **Bottom:** the multi-agent evolver constructs and refines a harness tree across cycles via four roles (Analyst $\rightarrow$ parallel Researchers $\rightarrow$ Builder $\rightarrow$ Verifier) with a persistent cross-cycle workspace and temporal-reveal feedback. **Top:** at solve time, a router agent reads each branch’s workspace via git show and routes the incoming task x_t to the most suitable branch. Two human-in-the-loop hooks (task-board steering and research-phase assistance) trigger only when the evolver’s history lacks relevant signal.

3.3 Sustained Auto-Harness via Multi-Agent Evolution

Reducing L_{evo} requires expanding the evolution capability Φ . We identify three structural limitations of prior single-agent evolvers: (i) evolution involves multiple complex subtasks that compete for a single context window, capping the complexity of constructible harnesses; (ii) the evolver lacks a proper feedback channel for unbounded streams where labels arrive asynchronously; (iii) without cross-cycle memory, the evolver’s construction capability remains static and cannot compound with the growing history induced by unbounded streams (D1). We address each limitation in our multi-agent evolver (Figure 3, bottom panel):

(1) Multi-agent with distinct roles and objectives. Because unbounded streams (D1) keep expanding the trajectory history an evolver must interpret, existing single-agent evolvers must fit analysis, research, implementation, and verification into one context window. We decompose evolution into four agents (Analyst $\rightarrow$ parallel Researchers $\rightarrow$ Builder $\rightarrow$ Verifier), each with a dedicated objective and full context budget. This eliminates the single-window bottleneck and lets parallel Researchers explore disjoint hypotheses without premature convergence.

(2) Temporal-reveal feedback. Under unbounded streams (D1), labels arrive asynchronously (e.g., a prediction market resolves days after the trade). We implement a *temporal-reveal gate* that surfaces each task’s ground-truth label to the evolver only after its resolution date, providing a proper streaming feedback signal without leaking future information.

(3) Persistent cross-cycle state. We provide the evolver with a dedicated workspace that persists across cycles, containing a task board (prioritised failure analysis), research logs (tested hypotheses with pass/fail verdicts), architecture documentation, and verification tests. This cross-cycle memory enables the evolver to refine its construction ability over time and build upon prior evolution experience rather than restarting from scratch.

3.4 Solve-Time Adaptation via Harness-Tree Routing

Reducing L_{adapt} requires the deployed harness to approximate $C_{\Phi}^*(x_t)$ across heterogeneous tasks. A naive approach would perform full adaptation at solve time, but this is expensive and unreliable under latency constraints. Therefore, we propose shifting the heavy adaptation work into evolution time, which requires the evolver to construct a *branching harness tree* of specialised harnesses, and leave only a lightweight routing decision at

solve time: selecting the most suitable harness branch given the current task context (Figure 3, top panel).

We instantiate this with two designs (Figure 3, top panel). **(1) Branching harness tree (built at evolution time).** The solver’s workspace is a git repository. The evolver constructs regime-specific branches (e.g., branch/crypto-classical, branch/binary-reversing) during evolution, each carrying its own prompt, skills, and tool registry; git provides versioning, isolation, and lineage tracking across branches. **(2) Agentic routing (executed at solve time).** A router agent reads each branch’s workspace via `git show` and selects the most suitable branch given x_t ’s context; the solver then checks out that branch and executes.

3.5 Human-in-the-Loop Channel

A failure mode orthogonal to the three deployment dimensions is *experience insufficiency*: some tasks require inputs the evolver cannot synthesize from $\mathcal{H}_t$ alone, such as API credentials, novel-language sources, or proprietary endpoints not encountered before. In this regime, neither a richer evolver class Φ nor solve-time routing can recover what is absent from history; the regret decomposition cannot be reduced further because the full-history utility itself is low. We therefore introduce a third axis outside $L_{\text{evo}} + L_{\text{adapt}}$: a human-in-the-loop channel that augments $\mathcal{H}_t$ with external input. Two structurally triggered hooks in the multi-agent pipeline (Figure 3) each provide a different form of human steering: **(1) Task-board steering.** After the Analyst updates the task board, a human may review it to add entries, adjust priorities, or supply domain guidance and source access. This proactively steers the subsequent research cycle with direction the evolver cannot derive from trajectories alone. **(2) Interactive assistance during research.** When a Researcher hits a barrier mid-execution that requires human intervention (e.g., an authentication wall), the hook prompts the human in real time. This reactively unblocks the research agent at the point of failure.

4 Experiments

4.1 Experimental Setup

Benchmarks. We evaluate on three open-ended task streams (Table 1): *PolyBench* for prediction markets (Cheng et al., 2026), *CTF-Dojo* for secu-

Table 1: Benchmark statistics for the three chronological task streams. Interval denotes the temporal ordering granularity used during evaluation.

Bench.	Tasks	Span	Interval	Domain
PolyBench	5,075	Feb 6–22, 2026	Daily	Prediction markets
CTF-Dojo	261	2011–2024	Year	Security
FutureX	503	Jan–Apr 2026	Weekly	Forecasting

rity challenges (Zhuo et al., 2025), and *FutureX* for event forecasting (Zeng et al., 2025). All three enforce strict temporal order and covers the three dimensions of challenge. Details and non-stationarity diagnostics are in Appendix A and B.2.

Solver and evolver. We use Claude Sonnet 4.6 as the solver for all experiments unless specified, and use Claude Opus 4.6 as the evolver, both at temperature 0 to attribute gains to the evolution algorithm rather than sampling noise; the no-evolution controls additionally report base-agent results for DeepSeek and Kimi. All algorithms share the same batch size (100/20/20 for PolyBench/CTF-Dojo/FutureX), batch loop, and temporal-reveal gate; only the evolution algorithm varies.

Metrics. Each benchmark reports metrics tailored to its deployment goal. PolyBench reports coverage, all-task accuracy, and total portfolio return. For CTF-Dojo and FutureX, we use the same stream metrics: pass rate, Sonnet-adjusted OLS trend over chronological task order, and Q25@60, the 25th percentile of 60-task sliding-window pass rates. Full definitions are in Appendix A. [#Zewen: need to come back later#](#)

Baselines. We compare against systems grouped as in Table 2. The no-evolution controls are base agents using *Sonnet* (), *DeepSeek* (), and *Kimi* (). We compare with five auto-harness baselines: *A-Evolve* (A-Evolve authors, 2025), *GEPA* (GEPA authors, 2025), *Meta-Harness* (Meta-Harness authors, 2026) (archive-based proposer, $k=1$), *Continual Harness*, and *SkilIOS*. We also compare with one human-designed system *OctoTools* (OctoTools authors, 2026).

Research questions. **RQ1** (§4.2): How does Adaptive Auto-Harness compare against existing auto-harness systems on open-ended task streams? **RQ2** (§4.3): How does Adaptive Auto-Harness address benchmark-specific bottlenecks? **RQ3** (§4.4): Does stateful multi-agent evolution with evaluation feedback provide additional gains? **RQ4** (§4.5): Can solve-time routing effectively leverage specialized harness branches? **RQ5** (§4.6): Can human

steering help when history provides insufficient signal?

4.2 Comparison with Baselines

#Zewen: revisit# Table 2 reports the main stream comparison. The central pattern is not that one baseline is weak, but that each baseline fails under a different deployment pressure. A-Evolve is competitive on the pass-rate streams, reaching 45.2 on CTF-Dojo and 47.5 on FutureX, yet it covers only 21.1% of PolyBench markets and therefore misses most trading opportunities. Meta-Harness is the strongest baseline on PolyBench return, but does not translate that advantage to CTF-Dojo or FutureX pass rates. OctoTools, the frozen human-designed system, is stable but lacks the stream-specific adaptation needed to lead any metric.

Our two variants cover complementary regimes. The multi-agent evolver is the strongest reliability-oriented system: it leads all three CTF-Dojo metrics, achieves the best FutureX pass rate, and has the strongest FutureX lower-tail stability. Navigation is most valuable when the evolved tree contains specialized branches whose choice affects payoff: it achieves the highest PolyBench return while retaining high coverage and accuracy, and ranks second on CTF-Dojo pass/Q25@60. The main exception is FutureX Trend, where Meta-Harness has the strongest late-stream slope and the multi-agent run regresses. This motivates reporting pass, Trend, and Q25@60 together: average success, improvement over time, and lower-tail robustness expose different failure modes.

Reading the table by benchmark makes the deployment pressure sharper. On PolyBench, the full evolver expands coverage to 95.8%, far above the strongest non-ours coverage result (55.3%), but navigation still slightly improves return because payoff depends on selecting the right specialized branch and confidence policy, not only answering more markets. On CTF-Dojo, the same multi-agent design is not merely higher on average: it also has the best upward Trend and best Q25@60, indicating that accumulated exploit patterns improve the stream rather than only solving early/easy tasks. FutureX is the stress case. The multi-agent system has the highest pass rate and lower-tail stability, while navigation and Meta-Harness have better Trend; this split is exactly why a single aggregate score would hide whether the bottleneck is construction, branch selection, or non-stationary source access.

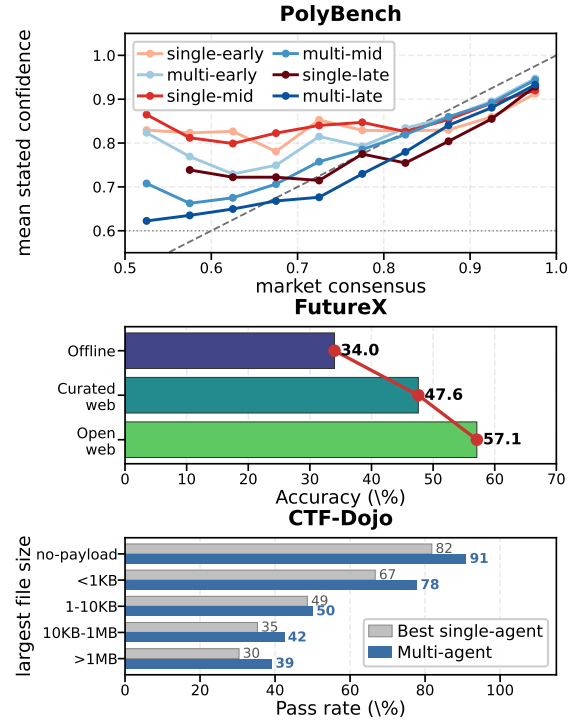


Figure 4: L_{evo} evidence across benchmark-specific bottlenecks. PolyBench stresses confidence calibration, FutureX stresses web-retrieval access, and CTF-Dojo stresses payload handling of different file sizes.

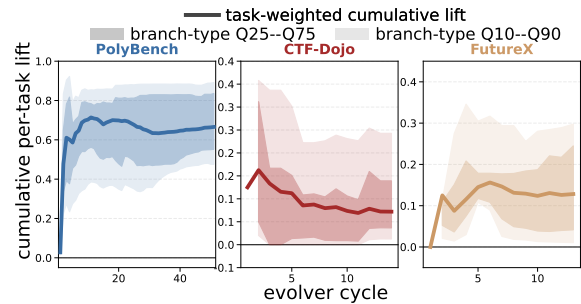


Figure 5: L_{adapt} evidence from branch specialization. Curves show cumulative routing lift over the baselines, while shaded bands show harness branch performance spread across cycles.

4.3 Benchmark Bottlenecks

RQ2 asks whether Adaptive Auto-Harness targets the bottlenecks that actually limit each benchmark. Figure 4 uses benchmark-specific stressors to expose the required constructed capabilities. On PolyBench, useful harnesses must calibrate confidence against market consensus: the multi-agent curves move upward with consensus, while weaker single-agent variants remain much flatter and under-use high-consensus opportunities. On FutureX, accuracy changes almost monotonically with retrieval

Table 2: Main comparison across no-evolution agents, auto-harness baselines, a human-designed system, and our two variants. Bold marks the best result per metric and underline marks second best.

Benchmark	Metric	No evolution				Auto-harness baselines				Human	Ours	
		Sonnet	DeepSeek	Kimi	A-Evolve	GEPA	Meta Harness	Cont. Harness	SkillIOS	OctoTools	Multi agent	Nav.
PolyBench	Coverage $\uparrow$	31.7	–	–	21.1	32.6	55.3	10.4	24.1	54.4	95.8	<u>91.4</u>
	Accuracy $\uparrow$	22.2	–	–	18.4	13.4	50.8	8.5	21.4	40.0	79.8	<u>77.4</u>
	Return $\uparrow$	+1.8	–	–	+7.2	+0.3	+320	+1.7	+3.6	+20.4	<u>+351</u>	+352
CTF-Dojo	Pass $\uparrow$	37.2	–	–	45.2	42.9	41.0	25.7	29.5	38.3	52.1	<u>46.0</u>
	Trend $\uparrow$	+0.0	–	–	<u>+30.4</u>	−3.1	+0.5	+13.1	+22.7	−11.4	+34.6	<u>+3.6</u>
	Q25@60 $\uparrow$	34.2	–	–	<u>38.3</u>	40.0	35.0	24.6	29.6	36.2	44.6	<u>42.9</u>
FutureX	Pass $\uparrow$	31.0	–	–	<u>47.5</u>	28.2	29.4	31.8	29.8	25.6	49.5	<u>44.1</u>
	Trend $\uparrow$	+0.0	–	–	<u>+3.3</u>	<u>+10.6</u>	+14.2	+9.0	+4.4	+7.7	−1.3	<u>+7.3</u>
	Q25@60 $\uparrow$	20.0	–	–	<u>37.9</u>	21.3	22.9	22.9	21.3	18.3	38.3	<u>35.0</u>

access, showing that forecasting quality depends on source acquisition and web-search tooling rather than reasoning alone. On CTF-Dojo, payload size exposes a different bottleneck: multi-agent pass rate drops from 91% on no-payload tasks to 39% above 1MB, and the best single-agent line has the same downward shape. These panels show that the system must construct different capabilities for different benchmarks: confidence calibration for markets, search/source tools for forecasting, and payload-handling infrastructure for security tasks.

Figure 5 shows the cumulative lift with adaptive-harness routing compared against the baselines and the quantile shows the performance across different branches. If a single committed harness were enough, routing lift over the median branch would collapse toward zero and branch-type quantiles would converge. Instead, lift remains positive across cycles, and the final-cycle branch-type IQRs stay wide: 28% on PolyBench, 10% on CTF-Dojo, and 20% on FutureX. The adaptation bottleneck is especially clear on PolyBench, where category- and payoff-sensitive branches produce persistent lift, and on FutureX, where the positive lift coexists with the main-table Trend failure.

4.4 Stateful Multi-Agent Evolution

RQ3 asks whether stateful multi-agent evolution with evaluation feedback provides additional gains. Figure 6 answers yes: compared with the full system, removing feedback collapses PolyBench CWR from 44.3 to 2.6, while removing memory lowers CTF-Dojo from 43% to 30% and FutureX from 44% to 38%. These drops show that the gains come from the stateful interface to the stream: delayed outcomes must enter later evolution steps,

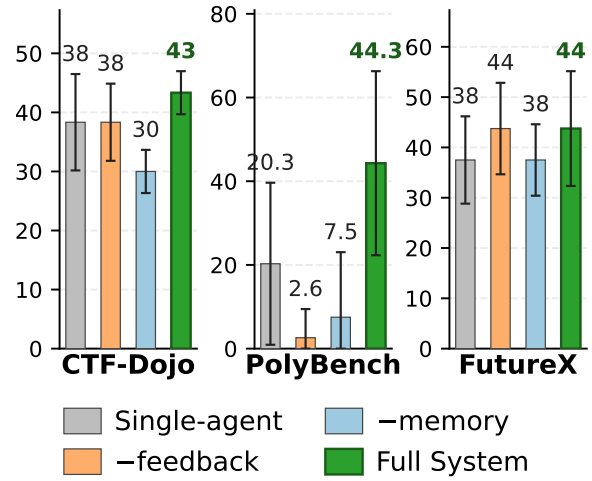


Figure 6: Ablations for multi-agent evolution. Removing evaluation feedback or cross-cycle memory degrades end-of-stream performance relative to the full stateful system.

and reusable exploit artifacts, parsing rules, and source heuristics must persist across cycles. Without these state channels, a multi-agent loop can still propose variants, but it cannot reliably turn stream evidence into stronger harnesses.

4.5 Solve-Time Routing on the Harness Tree

RQ4 asks whether solve-time routing can leverage the specialized branches created by evolution. To isolate routing from construction, all strategies use the same evolved harness tree (Figure 7; trace in Figure 8(b)): *Oracle* chooses the per-task best branch, *Adapt* uses our router, *Naive* always uses main, and *Worst* chooses the per-task worst branch. The answer is mostly yes: Oracle–Naive gaps show that branch specialization matters (37% on CTF-Dojo, +9 CWR on PolyBench, and 7%

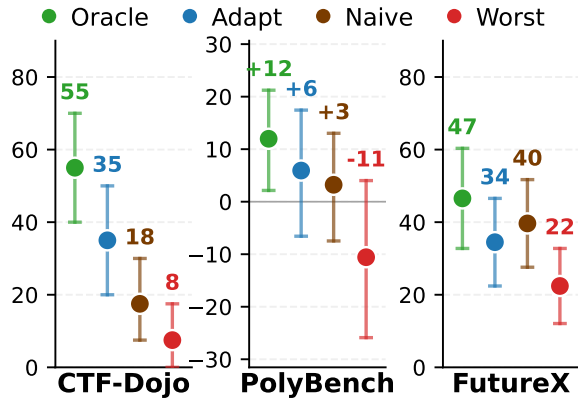


Figure 7: Solve-time routing evaluation on a evolved harness tree. Oracle reports the best branch performance per task, Adapt uses our agentic router, Naive always uses main workspace, and Worst reports the worst branch performance per task.

on FutureX), and Adapt converts this specialization into gains on CTF-Dojo (18% $\rightarrow$ 35%) and PolyBench (+3 $\rightarrow$ +6 CWR). FutureX is the exception: Adapt trails Naive (34% vs. 40%) even though Oracle reaches 47%, showing that useful branches exist but the router does not identify them reliably under fast source shifts.

4.6 Human Steering for Auto-Harnessing

RQ5 asks whether human steering helps when the stream history lacks the signal needed to build the next harness. We test this on FutureX because several failures come from missing search access or source-specific endpoint knowledge rather than from ordinary reasoning errors. Human input is therefore restricted to steering the auto-harness process: providing credentials when a research tool is blocked, or editing the task board with source directions that the evolver cannot infer from past trajectories. Figure 8(c) shows these two trigger types.

The answer is positive only when the steering targets the missing signal. In Figure 9, generic Polymarket transfer gives no lift, so the gains are not from adding arbitrary human advice. Broad search steering adds +5%, while task-board steering adds +20% on finance/technology and +15% on Western-specialty slices. These gains are localized to the slices where the missing source access or source map matters. Thus, human steering is useful for auto-harnessing when it supplies external information absent from history; it is not a substitute for labeling tasks or manually choosing branches.

5 Conclusions

Existing auto-harness systems neither improve construction over time nor adapt to each incoming task. We address both gaps with a stateful multi-agent evolver that accumulates cross-cycle knowledge and a harness-tree router that selects a suitable branch at solve time. On three finite prefixes of unbounded deployment streams, the system outperforms all baselines and is the only one whose capability compounds over the stream.

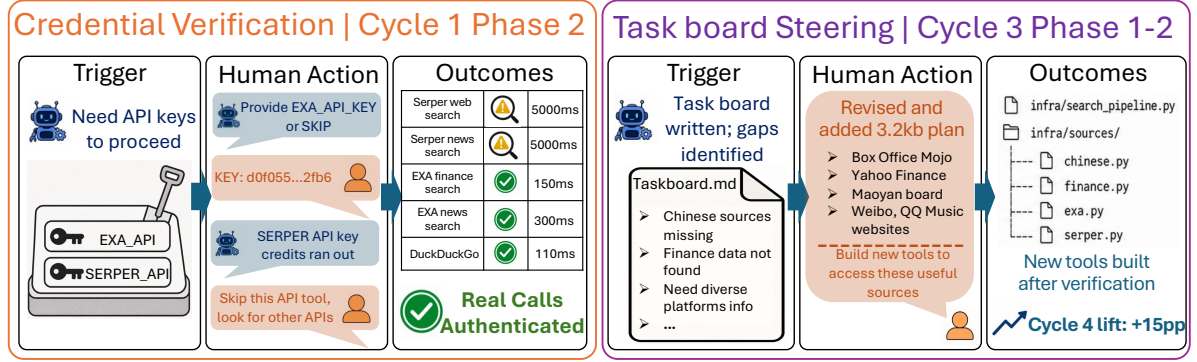
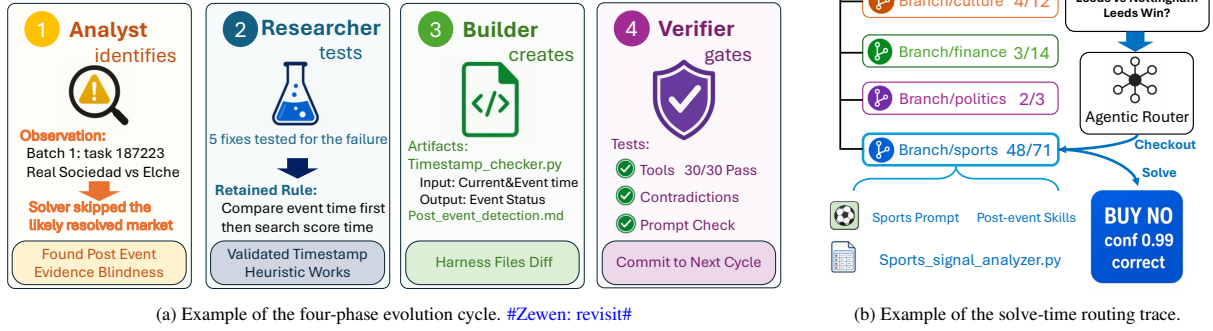


Figure 8: Extracted trajectories of the designed multi-agent evolution, agentic routing, and Human-in-the-Loop.

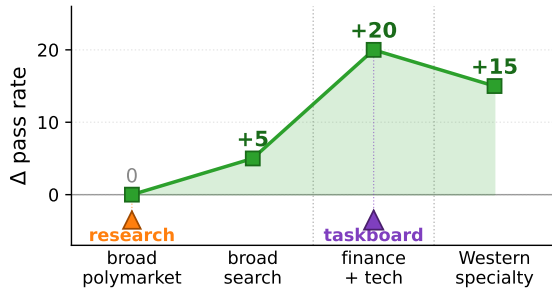


Figure 9: FutureX pass-rate lift with human steering. HITL is triggered twice at different cycle phases (Research phase at cycle 1; Analyze phase at cycle 3). Cycle 1 starts with the zero lift and then improves on later cycles. The detailed trajectories are recorded in Figure 8.

References

- A-Evolve authors. 2025. A-evolve: Linear-chain self-evolving agents. Placeholder citation; replace with canonical entry.
- Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, and 1 others. 2025. Gepa: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*.
- Shai Ben-David, John Blitzer, Koby Crammer, Alex Kulesza, Fernando Pereira, and Jennifer Wortman Vaughan. 2010. A theory of learning from different domains. *Machine Learning*, 79:151–175.
- Pietro Buzzega, Matteo Boschini, Angelo Porrello, Davide Abati, and Simone Calderara. 2020. Dark experience for general continual learning: A strong, simple baseline. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 15920–15930.
- Pu Cheng, Juncheng Liu, and Yunshen Long. 2026. Polybench: Benchmarking llm forecasting and trading capabilities on live prediction market data. *arXiv preprint arXiv:2604.14199*.
- Yaroslav Ganin, Evgeniya Ustinova, Hana Ajakan, Pascal Germain, Hugo Larochelle, François Laviolette, Mario Marchand, and Victor Lempitsky. 2016. Domain-adversarial training of neural networks. *Journal of Machine Learning Research*, 17(59):1–35.
- GEPA authors. 2025. Gepa: Reflective pareto prompt evolution. In *Conference on Neural Information Processing Systems (NeurIPS)*. Placeholder citation; replace with canonical entry.
- Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. 1991. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, volume 2024, pages 54107–54157.
- Seth Karten, Joel Zhang, Tersoo Upaa Jr, Ruirong Feng, Wenzhe Li, Chengshuai Shi, Chi Jin, and Kiran Vohra. 2026. Continual harness: Online adaptation for self-improving foundation agents. *arXiv preprint arXiv:2605.09998*.
- Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. 2026. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*.
- Jian Liang, Dapeng Hu, and Jiashi Feng. 2020. Do we really need to access the source data? source hypothesis transfer for unsupervised domain adaptation. In *International Conference on Machine Learning (ICML)*.
- Minhua Lin, Hanqing Lu, Zhan Shi, Bing He, Rui Mao, Zhiwei Zhang, Zongyu Wu, Xianfeng Tang, Hui Liu, Zhenwei Dai, and 1 others. 2026. Position: Agentic evolution is the path to evolving llms. *arXiv preprint arXiv:2602.00359*.
- Meta-Harness authors. 2026. Meta-harness: Archive-based harness search. Placeholder citation; replace with canonical entry.
- Rui Miao, Babak Shahbaba, and Annie Qu. 2025. Reinforcement learning for individual optimal policy from heterogeneous data. *Annals of statistics*, 53(4):1513.
- OctoTools authors. 2026. Octotools: A training-free planner-executor-tool-cards generalist. In *Annual Meeting of the Association for Computational Linguistics (ACL)*. Placeholder citation; replace with canonical entry.
- Siru Ouyang, Jun Yan, Yanfei Chen, Rujun Han, Zifeng Wang, Bhavana Dalvi Mishra, Rui Meng, Chunliang Li, Yizhu Jiao, Kaiwen Zha, and 1 others. 2026. Skillos: Learning skill curation for self-evolving agents. *arXiv preprint arXiv:2605.06614*.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*.
- Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. 2021. Tent: Fully test-time adaptation by entropy minimization. In *International Conference on Learning Representations (ICLR)*.
- Zifeng Wang, Zizhao Zhang, Sayna Ebrahimi, Ruoxi Sun, Han Zhang, Chen-Yu Lee, Xiaoqi Ren, Guolong Su, Vincent Perot, Jennifer G. Dy, and Tomas Pfister. 2022a. Dualprompt: Complementary prompting for rehearsal-free continual learning. In *European Conference on Computer Vision (ECCV)*, pages 631–648.
- Zifeng Wang, Zizhao Zhang, Chen-Yu Lee, Han Zhang, Ruoxi Sun, Xiaoqi Ren, Guolong Su, Vincent Perot, Jennifer G. Dy, and Tomas Pfister. 2022b. Learning to prompt for continual learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 139–149.
- Zhiyuan Zeng, Jiashuo Liu, Siyuan Chen, Tianci He, Yali Liao, Yixiao Tian, Jinpeng Wang, Zaiyuan Wang, Yang Yang, Lingyue Yin, and 1 others. 2025. Futurex: An advanced live benchmark for llm agents in future prediction. *arXiv preprint arXiv:2508.11987*.
- Terry Yue Zhuo, Dingmin Wang, Hantian Ding, Varun Kumar, and Zijian Wang. 2025. Training language model agents to find vulnerabilities with ctf-dojo. *arXiv preprint arXiv:2508.18370*.

A Benchmark Details

We provide full descriptions of the three benchmarks summarized in §4.1. Across all three, tasks arrive in strict chronological order: the solver may only access information available before each task’s release timestamp, enforced by a temporal-reveal gate that filters retrieval results and ground-truth labels accordingly. Together, the benchmarks instantiate the deployment dimensions defined in §1: finite evaluated prefixes of unbounded streams (D1), within-stream heterogeneity across topics, languages, and challenge categories (D2), and distribution shift over time (D3). Non-stationarity diagnostics are shown in Appendix B.2. Throughout, let N be the number of tasks in the stream, $\mathbb{K}_i$ the correctness indicator on task x_i , and $\hat{p}_i \in [0, 1]$ the solver’s confidence.

A.1 PolyBench

Source and composition. PolyBench is a Polymarket-derived prediction-market stream of 5,075 binary-outcome trades over 16 days (Feb 6–22, 2026), spanning politics, sports, finance, crypto, and entertainment markets. Markets are sampled to keep the per-day volume balanced across the five categories; resolution uses the official Polymarket outcome reported on-chain.

Temporal-reveal gate. For each market we record its release timestamp and resolution timestamp. The solver may use any retrieval result whose source publication time precedes the release timestamp; resolution outcomes are revealed to the evolver only after the resolution date.

Metrics. Let τ be the confidence gate (default $\tau=0.55$).

$$\text{Cov} = \frac{1}{N} \sum_i \mathbb{K}_i [\hat{p}_i > \tau]$$

$$\text{Acc} = \frac{1}{N} \sum_i \mathbb{K}_i$$

$$\text{CWR} = \sum_i w_i (2\mathbb{K}_i - 1), \quad w_i \propto |\hat{p}_i - 0.5|$$

Total portfolio return is reported as the dollar P&L of a fixed-bankroll trader sized proportionally to w_i on each gated trade.

A.2 CTF-Dojo

Source and composition. CTF-Dojo is a 261-challenge security-competition stream drawn from pwncollege/ctf-archive, chronologically ordered over 13 years (2011–2024). Challenges span binary exploitation, web security, cryptography, reverse engineering, and forensics categories. The

chronological release order intentionally exposes the solver to evolving CTF tooling and trick families across the 13-year span (D3).

Sandbox and verification. Each challenge runs inside a per-task Docker sandbox with a constrained network policy; flags are submitted as text and verified via SHA-256 hash comparison against the official flag.

Metrics.

$$\text{Pass} = \frac{1}{N} \sum_i \mathbb{K}_i$$

$$\text{Trend} = \text{slope}_{\text{OLS}} \left(\{(t_i, \mathbb{K}_i - \mathbb{K}_i^{\text{Sonnet}})\}_{i=1}^N \right)$$

$$\text{Q25}_{60} = Q_{0.25} \left(\left\{ \frac{1}{60} \sum_{i \in W_j} \mathbb{K}_i \right\}_j \right)$$

where t_i is the normalized chronological task index and W_j ranges over 60-task sliding windows. Trend measures whether a method’s task-level advantage over the Sonnet no-evolution control improves over the stream; the Sonnet row is therefore zero by construction. Q25@60 reports the lower-tail sliding-window pass rate, so a method must avoid weak windows rather than only score well on average.

A.3 FutureX

Source and composition. FutureX is a 503-question event-forecasting stream over 82 days (Jan–Apr 2026) drawn from FutureX-Past. Questions cover finance, technology, geopolitics, and entertainment domains, with both English and Chinese language variants. The zh-finance slice requires Chinese-language sources unavailable through default English-only retrieval.

Retrieval modes. *Strict* mode resolves all retrieval through the Wikipedia revision API restricted to revisions before the task’s release date. *Live* mode uses DuckDuckGo with a hard cutoff filter applied to result timestamps. The human-steering experiment in RQ5 uses live mode because authenticated providers such as Exa and Serper are gated behind credentials the evolver cannot synthesize from history.

Metrics.

$$\text{Pass} = \frac{1}{N} \sum_i \mathbb{K}_i$$

$$\text{Trend} = \text{slope}_{\text{OLS}} \left(\{(t_i, \mathbb{K}_i - \mathbb{K}_i^{\text{Sonnet}})\}_{i=1}^N \right)$$

$$\text{Q25}_{60} = Q_{0.25} \left(\left\{ \frac{1}{60} \sum_{i \in W_j} \mathbb{K}_i \right\}_j \right)$$

These are the same aligned stream metrics used for CTF-Dojo. Trend is Sonnet-adjusted at the task

level, and Q25@60 captures lower-tail deployment reliability across the chronological stream.

B Additional Figures

B.1 Experiment Mechanism Figures

B.2 Benchmark Non-Stationarity Diagnostics

Figures 13–15 provide benchmark-specific evidence that the temporal streams are not static IID pools.

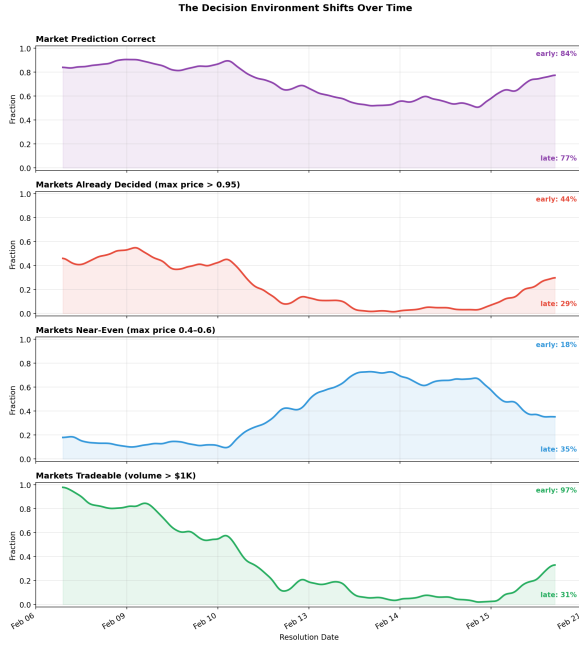


Figure 13: **PolyBench non-stationarity.** Strategy-relevant market properties vary across the temporal stream, so a harness fitted to earlier batches can become mismatched to later markets.

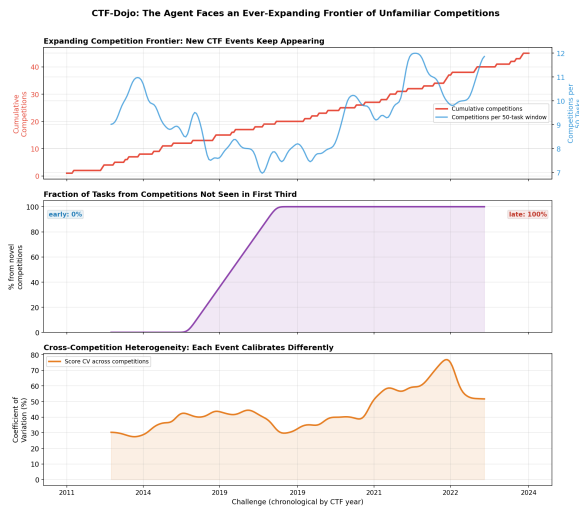


Figure 14: **CTF-Dojo non-stationarity.** The chronological stream expands into new competition frontiers, exposing the solver to changing challenge regimes rather than a fixed IID task pool.

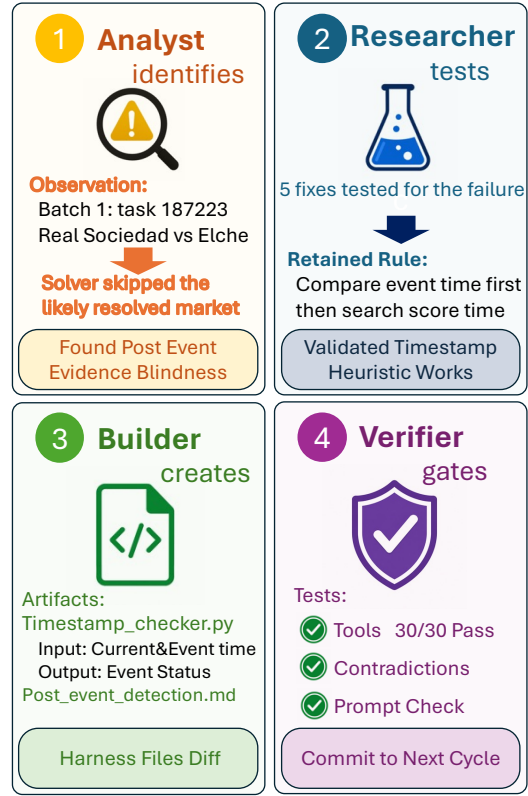


Figure 10: Representative sustained-evolution cycle. The Analyst diagnoses a failure, Researchers test candidate fixes, the Builder creates reusable artifacts, and the Verifier gates the change before the next cycle.

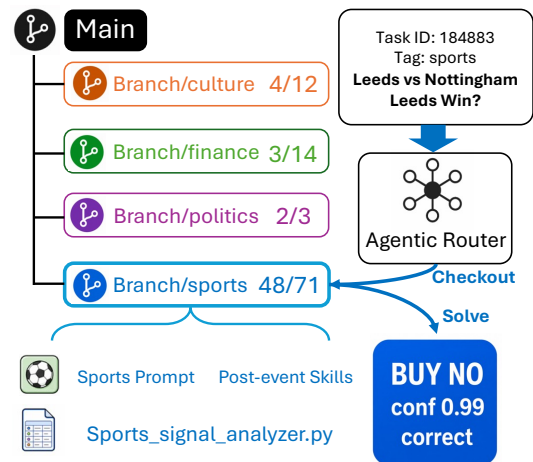


Figure 11: Representative solve-time routing trace. The router reads branch metadata, selects the sports-specialized branch for a sports forecasting task, and solves with the branch-specific prompt, skills, and tools.



Figure 15: **FutureX non-stationarity.** Language, source, and difficulty properties shift across batches, creating solve-time harness mismatch when one static harness is reused for all tasks.

C Limitations

Benchmark coverage. We evaluate three domains: prediction markets, cybersecurity challenges, and event forecasting. The D1/D2/D3 frame should be tested further on additional deployment streams such as robotics, biomedical reasoning, and long-form writing.

Evolver class is not scalar. Our L_{evo} argument is diagnostic, not predictive. The multi-agent evolver’s empirical advantage is demonstrated via ablations and benchmark bottleneck analyses, not by a formal class-expansion bound.

Human steering cost model. We measure human steering by invocation and resulting slice-level lift. In production, organizational cost, specialist availability, and response-time guarantees may dominate the raw time spent per intervention.

D Ethics Statement

We use benchmark tasks derived from public or research datasets and do not introduce private user data. PolyBench uses public prediction-market questions and outcomes; FutureX uses event-forecasting questions and time-filtered retrieval; CTF-Dojo uses public CTF challenges executed in isolated Docker sandboxes. The CTF experiments are limited to challenge containers and flag verification, and do not target real systems or third-party infrastructure.

The human-steering experiments are conducted by the authors and are limited to source guidance, task-board edits, and credential decisions for search tools. We do not ask humans to label benchmark answers or manually choose solver branches. Credentials and provider-specific secrets are not reported

in the paper or figures. Because auto-harness systems can strengthen autonomous agents, especially on cybersecurity tasks, releases should preserve sandbox restrictions, avoid exposing live credentials, and document that CTF results are for authorized benchmark settings only.

E Prompt Interfaces

E.1 F_Navigate System Prompt

The navigation prompt instructs the router to inspect the task context and branch metadata, compare candidate harness branches, and select exactly one branch before solving. The router may read branch prompts, tools, skills, and summaries, but it does not mutate the harness during solve time.

E.2 Analyst, Researcher, Builder, and Verifier Prompts

The multi-agent evolution prompts assign four separated roles. The Analyst summarizes failures and updates the task board; Researchers test hypotheses and gather evidence; the Builder converts verified findings into prompts, tools, skills, or infrastructure; and the Verifier runs checks that gate whether the change is committed to the next cycle.

E.3 Human-Steering Trigger Prompts

The human-steering prompts are used only when the evolver lacks information that cannot be inferred from $\mathcal{H}_t$, such as blocked credentials or source-specific access paths. The prompt requests the minimum external input needed to continue evolution and records the supplied steering in the task board or research log.

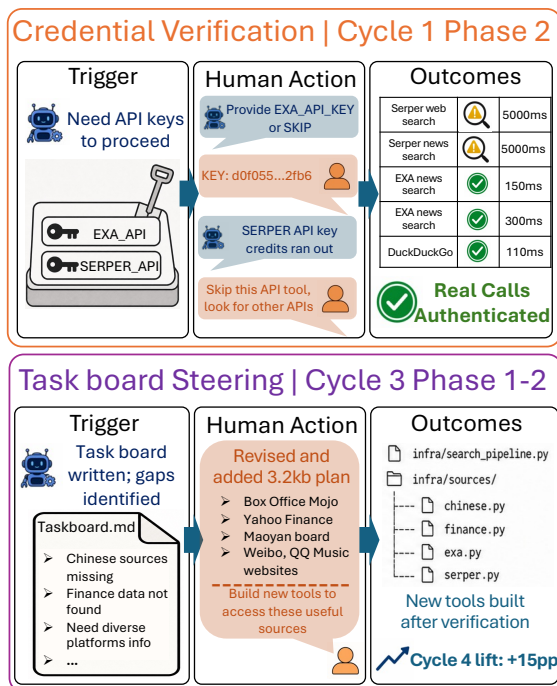


Figure 12: Representative human-steering triggers. Credential verification unlocks authenticated search during research, while task-board steering supplies source directions before the next build.